

Linjär Algebra med fokus på 3D-matematik

Uppgift T01 – Vektorklasser

Uppgiftens målsättning

Genom att bygga en egen vektorklass som template får den studerande en förståelse för matematiken för vektorer och träna på att översätta matematiska begrepp till kod. Utöver det får den studerande öva på att göra templates och operatoröverlagring.

Uppgift

Testprojekt

Ni får en test-suite **på torsdag**, som testar implementationen. Fokus för er i den här uppgiften är att implementera funktionaliteten till de interface ni fått i vektorklasserna och att själva skriva enhetstesterna.

När ni får tillgång till våra tester på torsdag, se då till att er implementation går igenom våra tester, det är en förutsättning för att få uppgiften godkänd! Om er kod inte går igenom något av våra tester, jämför då era egna tester med våra för att få en djupare förståelse så att ni hittar problemet.

Tänk på att template-kod som inte används kompileras inte alls, så all kod måste användas för att kompilatorn ska kunna hitta fel i den. Glöm därför inte att testa de typer som det är krav på att de stöds.

Vektor

I mappen Student Code har ni fått tillgång till tre filer med tomma klasstemplates: **Vector2**, **Vector3** och **Vector4**. Allt skall ligga i namespace **CommonUtilities** och klasserna är templatiserade och tar därför ett typargument: typen för komponenterna.

Vector2<float> är alltså en **Vector2** där x- och y-medlemmarna är av typen **float**. **Vector3<int>** är en **Vector3** där x-, y- och z-medlemmarna är av typen **int**.

Varje klasstemplate ligger i en egen fil: **Vector2.h**, **Vector3.h** och **Vector4.h**. Utöver detta skall ytterligare en fil finnas som heter **Vector.h**. Den är där av praktiska skäl och inkluderar de övriga

tre filerna, finns behovet av alla vektorer råcker det således att inkludera denna. Eftersom de är templateklasser så skrivs koden direkt i header-filerna.

Funktionerna ska fungera *åtminstone* med typerna float, double och int. Dock är Normalize()- och Length()-funktionerna inexakta med int, vilket är ok. Däremot ska LengthSqr() fungera!

Vector3 Interface

```
namespace Tga
{
    template <typename T>
    class Vector3;
}
namespace CommonUtilities
{
    template <typename T>
    class Vector3
    {
    public:
        T x; //Note: this variable is explicitly public.
        T y; //Note: this variable is explicitly public.
        T z; //Note: this variable is explicitly public.

        //Creates a null-vector
        Vector3();

        //Creates a vector (aX, aY, aZ)
        Vector3(const T& aX, const T& aY, const T& aZ);

        //Copy constructor (compiler generated)
        Vector3(const Vector3<T>& aVector) = default;

        //Assignment operator (compiler generated)
        Vector3<T>& operator=(const Vector3<T>& aVector3) = default;

        //Destructor (compiler generated)
        ~Vector3() = default;

        // Returns a CommonUtilities Vector3 copy with another datatype as elements,
        // ex: converts from Vector3<int> to Vector3<float>
        // useful to explicitly do operations, such as addition, for an explicit type
        template<class TargetType>
        Vector3<TargetType> ToType() const;

        //Returns a copy of the vector as a Tga vector, use to interface with TGE
        Tga::Vector3<T> ToTga() const;

        //Returns a negated copy of the vector
        Vector3<T> operator-() const;

        //Returns the squared length of the vector, optimization compared to length
        T LengthSqr() const;

        //Returns the length of the vector, int vector is not required to work
```



```

    T Length() const;

    //Returns a normalized copy of this vector. Need not function for int vectors
    //Handle normalization of zero-vector by returning the zero vector

    Vector3<T> GetNormalized() const;

    //Normalizes the vector. Need not function for int vectors
    //Handle normalization of zero-vector by not modifying the vector.
    void Normalize();

    //Returns the dot product of this and aVector
    T Dot(const Vector3<T>& aVector) const;

    //Returns the cross product of this and aVector. Only for Vector3
    Vector3<T> Cross(const Vector3<T>& aVector) const;
};

//Returns the vector sum of aVector0 and aVector1
template <typename T>
Vector3<T> operator+(const Vector3<T>& aVector0, const Vector3<T>& aVector1);

//Returns the vector difference of aVector0 and aVector1
template <typename T>
Vector3<T> operator-(const Vector3<T>& aVector0, const Vector3<T>& aVector1);

//Returns the vector aVector0 component-multiplied by aVector1
template <typename T>
Vector3<T> operator*(const Vector3<T>& aVector0, const Vector3<T>& aVector1);

//Returns the vector aVector multiplied by the scalar aScalar. Vector * Scalar
template <typename T>
Vector3<T> operator*(const Vector3<T>& aVector, const T& aScalar);

//Returns the vector aVector multiplied by the scalar aScalar. Scalar * Vector
template <typename T>
Vector3<T> operator*(const T& aScalar, const Vector3<T>& aVector);

//Returns the vector aVector divided by the scalar aScalar
template <typename T>
Vector3<T> operator/(const Vector3<T>& aVector, const T& aScalar);

//Equivalent to setting aVector0 to (aVector0 + aVector1)
template <typename T>
void operator+=(Vector3<T>& aVector0, const Vector3<T>& aVector1);

//Equivalent to setting aVector0 to (aVector0 - aVector1)
template <typename T>
void operator-=(Vector3<T>& aVector0, const Vector3<T>& aVector1);

//Equivalent to setting aVector to (aVector * aScalar)
template <typename T>
void operator*=(Vector3<T>& aVector, const T& aScalar);

//Equivalent to setting aVector to (aVector / aScalar)
template <typename T>

```

```
void operator/=(Vector3<T>& aVector, const T& aScalar);

//Implementations below this line-----
}
```

Vector2 & Vector4 Interface

Koden för Vector2 samt Vector4 ser likadan ut som Vector3 med följande skillnader:

- Konstruktörerna tar två argument för Vector2 och fyra argument för Vector4
- Cross()-funktionen finns ej i Vector2 och Vector4
 - eftersom den endast existerar för 3 dimensioner
- Vector2 har ingen medlemsvariabel z
- Vector4 har en ytterligare medlemsvariabel w

Tips

Tester:

- Skriv tester innan ni skriver vektor funktionen, läs eller tänk ut vilket resultat som förväntas.
- Använd övningsuppgifterna för facit på resultat som du kan skriva in i expected.
- En tumregel är att skriva en testfunktion för varje funktion som ska testas, testa även vektorerna med olika datatyper som med **float, double och int**.
- För att testa floats och double har **Assert::AreEqual** överlagringar som tar in en tolerans testa med en tolerans på 0.00001 för dessa.

Vektor:

- Vektorerna har ett undantag för att få ha publika medlemsvariabler. Framtida klasser ska fortfarande ha privata variabler.
- Implementationen av templateklassernas medlemsfunktioner ska vara **inline**.
- Om en parameter inte ändras av funktionen ska den skickas in som **const**.
- Om en funktion inte ändrar på några medlemsvariabler ska funktionen vara **const**. Ändrar funktionen på medlemsvariabler så fundera en extra gång på om den verkligen behöver göra det – gäller speciellt operatorerna. Undvik att ändra på medlemsvariabler om ni inte måste.

- Er kod kommer att testas av ett automatiskt program, så det är viktigt att ni följer funktionssignaturerna i exemplet ovan exakt. Det innebär att namnen, antalet argument och deras typer måste stämma – inklusive const! Notera även att allt är omslutet av namespace `CommonUtilities`, vilket även det måste stämma. Det enklaste är att helt enkelt kopiera in exempelkoden och sedan fylla i funktions-kropparna med korrekt funktionalitet (kommentarerna är ok att plocka bort).
- För att testa konverterfunktionen till TGA vektorn på enklaste sätt kan finns en Tga vektor dummy i era testprogram:

```

Namespace Tga
{
    template <typename T>
    class Vector3
    {
    public:
        T x;
        T y;
        T z;
        //Creates a null-vector
        Vector3() : x(T(0)), y(T(0)), z(T(0)) {}
        //Creates a vector (aX, aY, aZ)
        Vector3(const T& aX, const T& aY, const T& aZ):x(aX), y(aY), z(aZ) {}
    };
}

```

Förslag om man vill gå djupare:

- Skapa extra hjälpfunktioner som konverterar mellan olika dimentionalitet
- Överlagra ostream << operatoren i vektor filerna så du enkelt kan skriva ut dem

```

template<typename T>
inline std::ostream& operator<<(std::ostream& aStream, const Vector3<T>& aVector)
{
    // TODO: Gather ostream print code, end with returning aStream
}

```